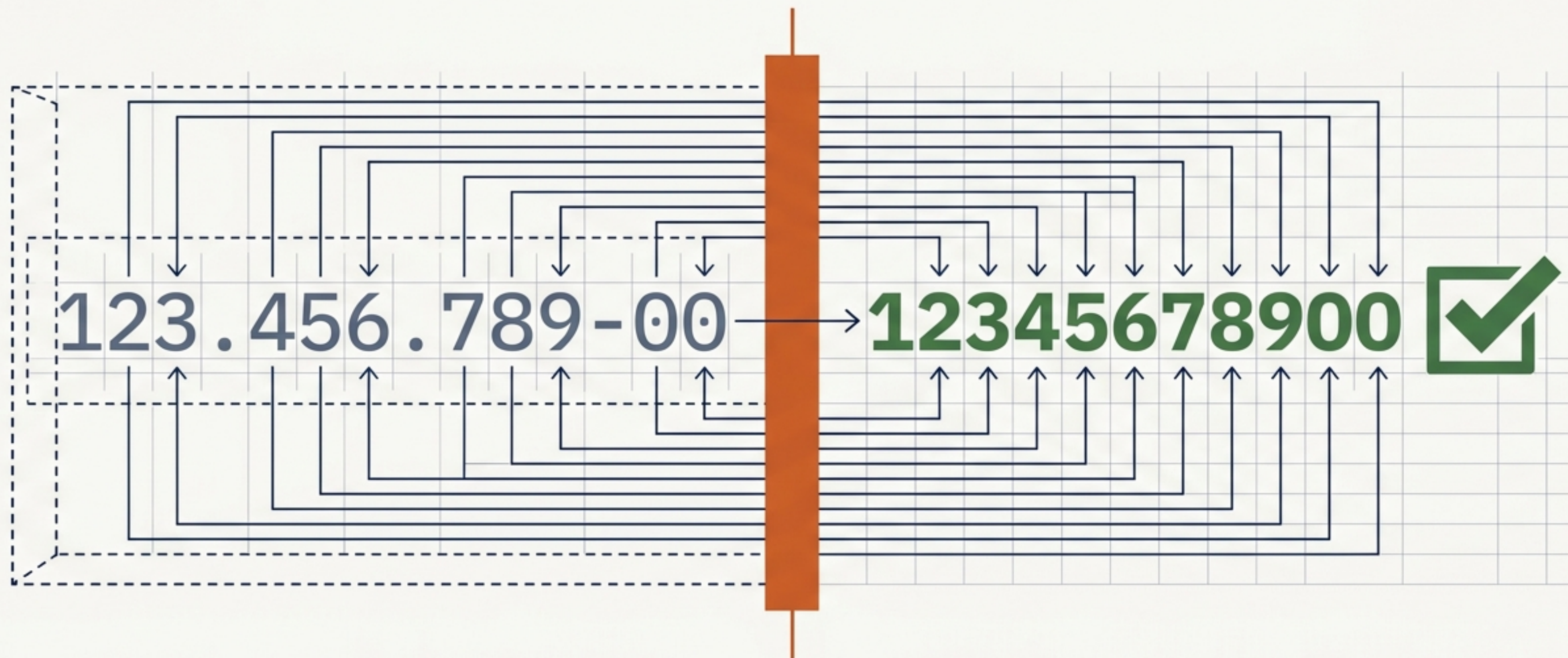


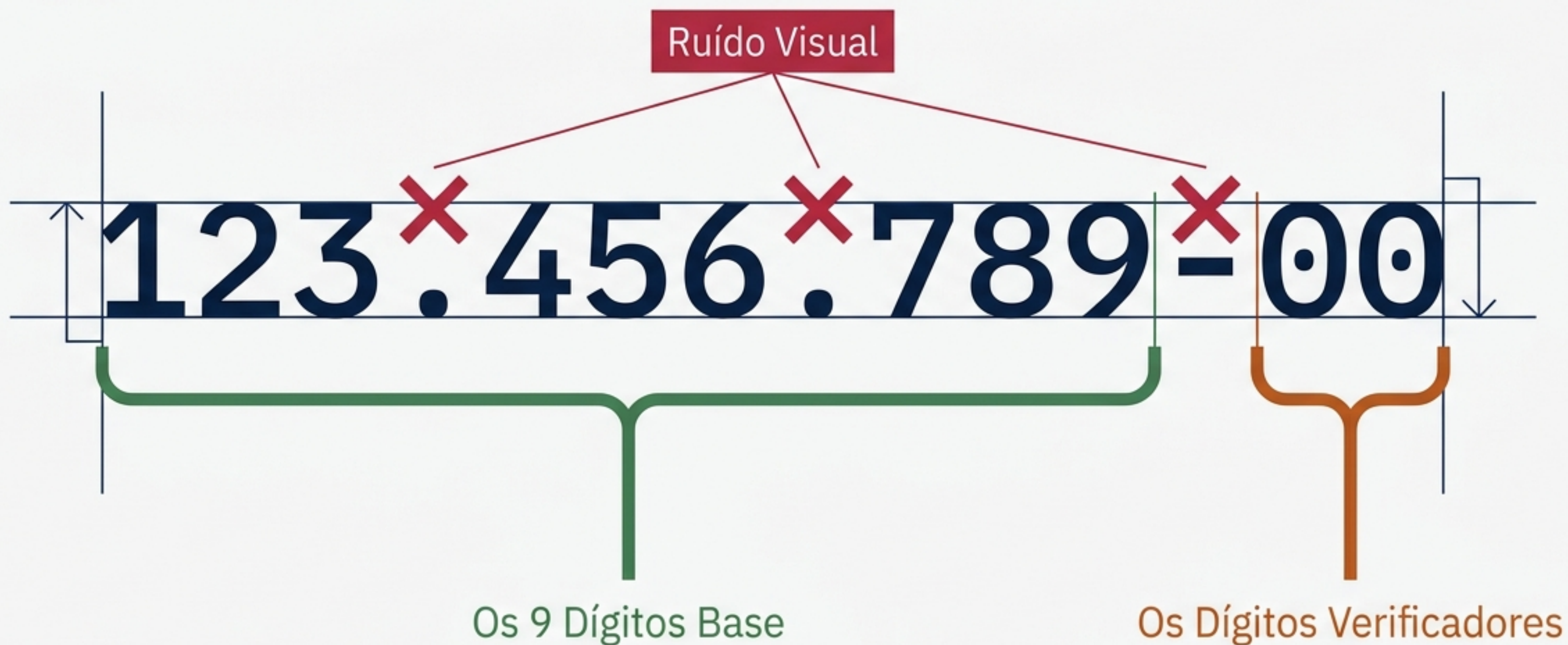
A Anatomia de um Validador de CPF em Python

Desconstruindo a lógica de sanitização e verificação matemática.



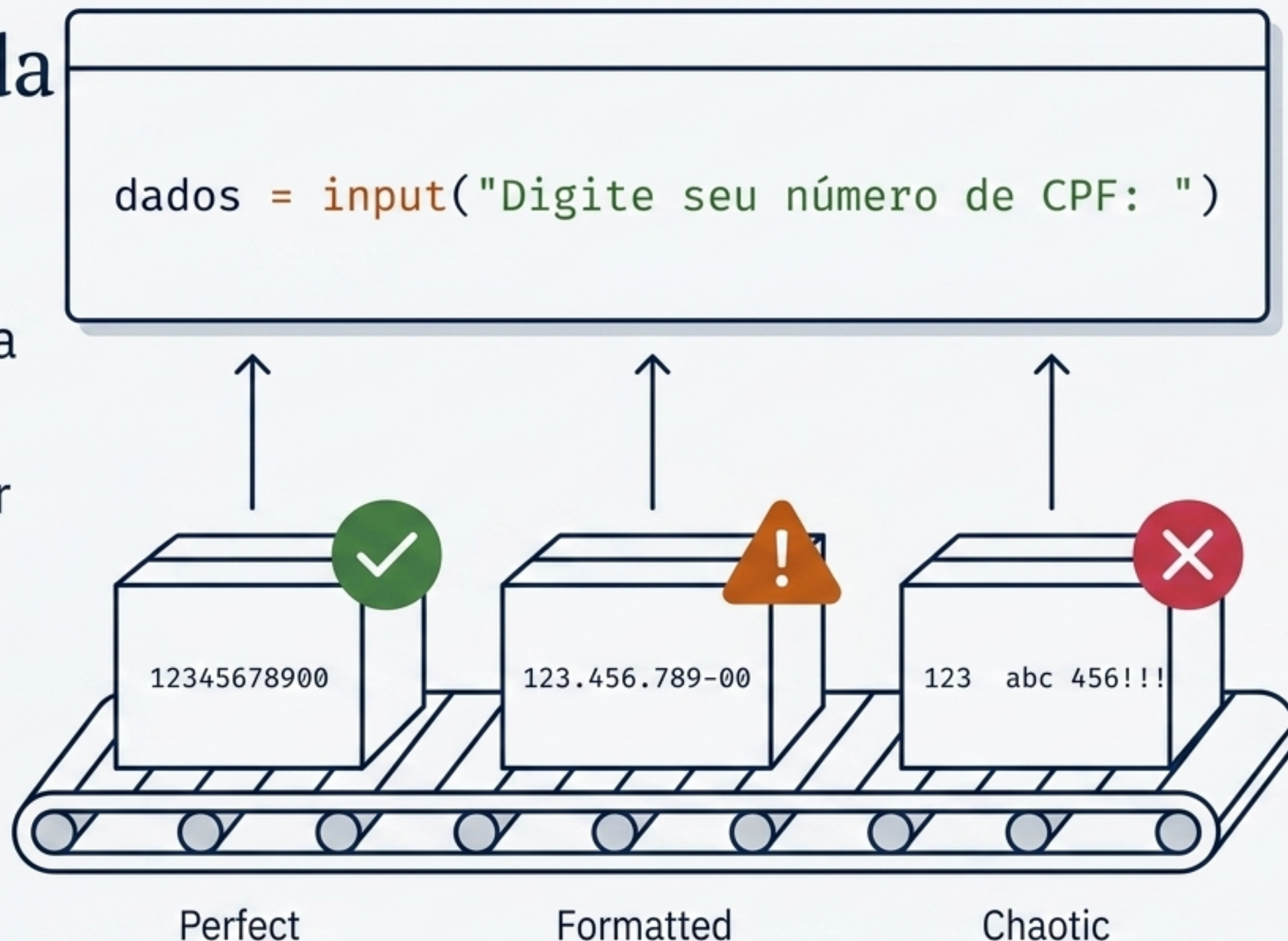
O Alvo da Validação

Um CPF é mais do que uma string aleatória; é uma equação embutida em 11 dígitos. O objetivo do script é provar que os dois últimos dígitos são o resultado matemático exato dos nove primeiros.



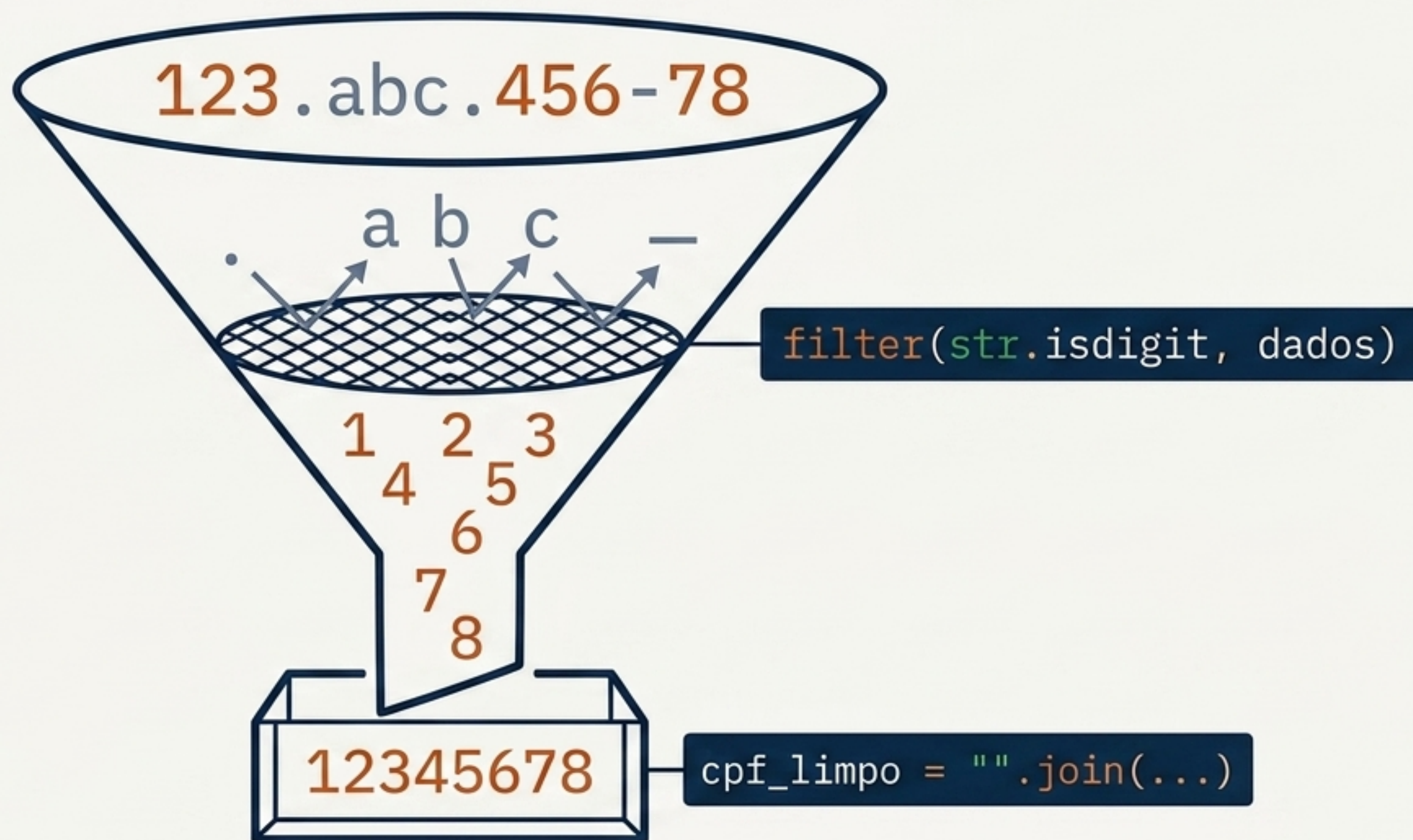
A Porta de Entrada

O ciclo de vida da validação começa capturando a entrada bruta do usuário. Neste estágio, não podemos confiar nos dados; eles podem conter espaços, letras ou formatação incorreta.



O Funil de Sanitização

O algoritmo extrai apenas *o que importa*. Usando uma função de filtro embutida, varremos a string para preservar exclusivamente caracteres numéricos, unindo-os novamente em uma string limpa.



O Primeiro Guardião: Comprimento

Antes de gastar processamento com matemática complexa, o sistema realiza uma verificação de falha rápida. Se a string limpa não tiver exatamente 11 posições, a execução é interrompida imediatamente.

```
if len(cpf_limpo) == 11:
```



A Armadilha da Multiplicação de Strings

Sequências repetidas passam na verificação matemática do Módulo 11, gerando falsos positivos. O Python previne isso elegantemente multiplicando o primeiro caractere por 11 para ver se ele forma a string inteira.

```
and cpf_limpo != cpf_limpo[0] * 11:
```



A Sala do Motor: Preparando o Cálculo

Com os dados validados estruturalmente, o script invoca a função central. Ela recebe dois parâmetros vitais: os dígitos a serem testados e os pesos decrescentes necessários para a equação.

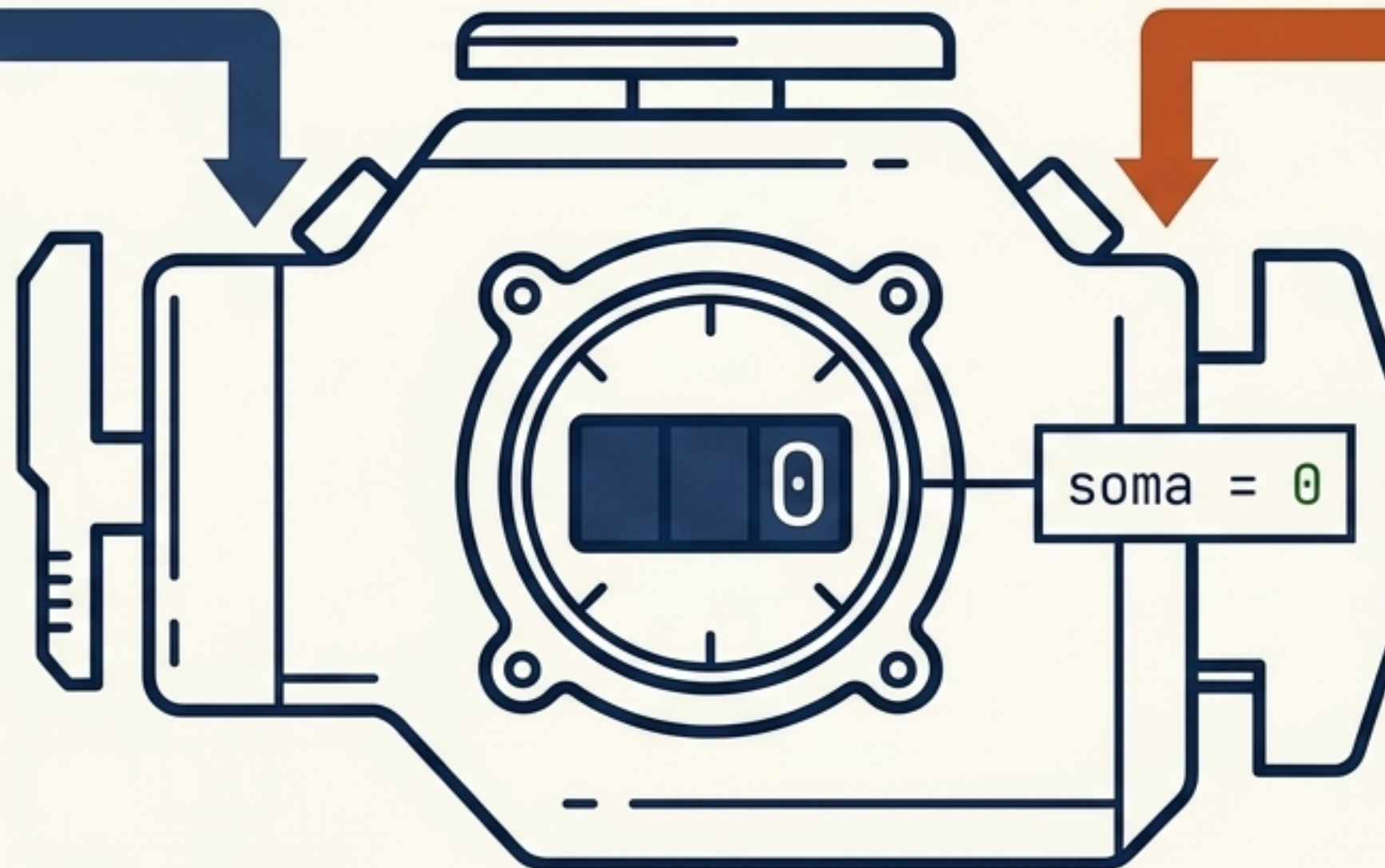
```
def digito_verificador(digitos, pesos):
```

digitos

['1', '2', '3']

pesos

[10, 9, 8]

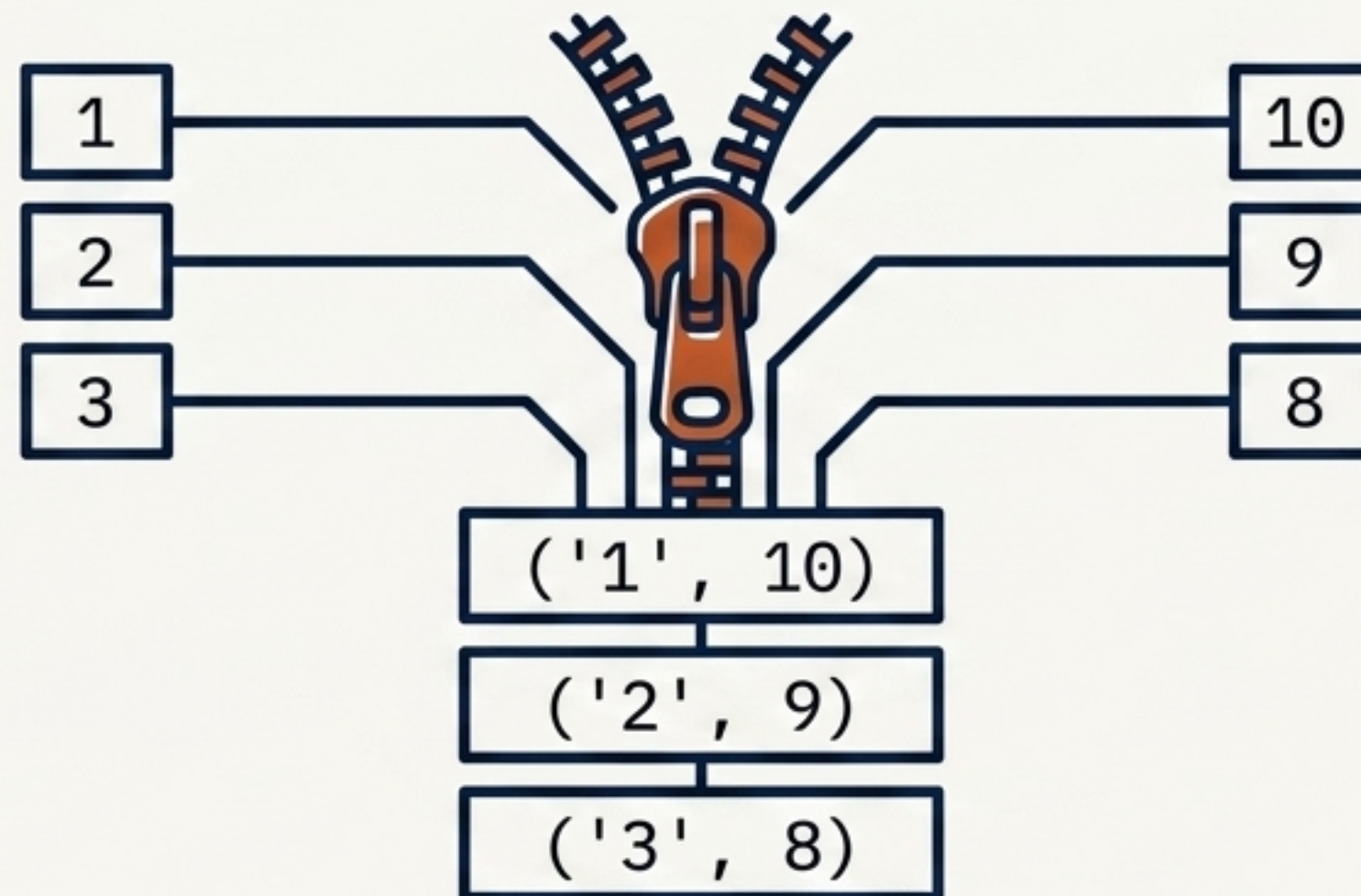


soma = 0

O Processo de Alinhamento

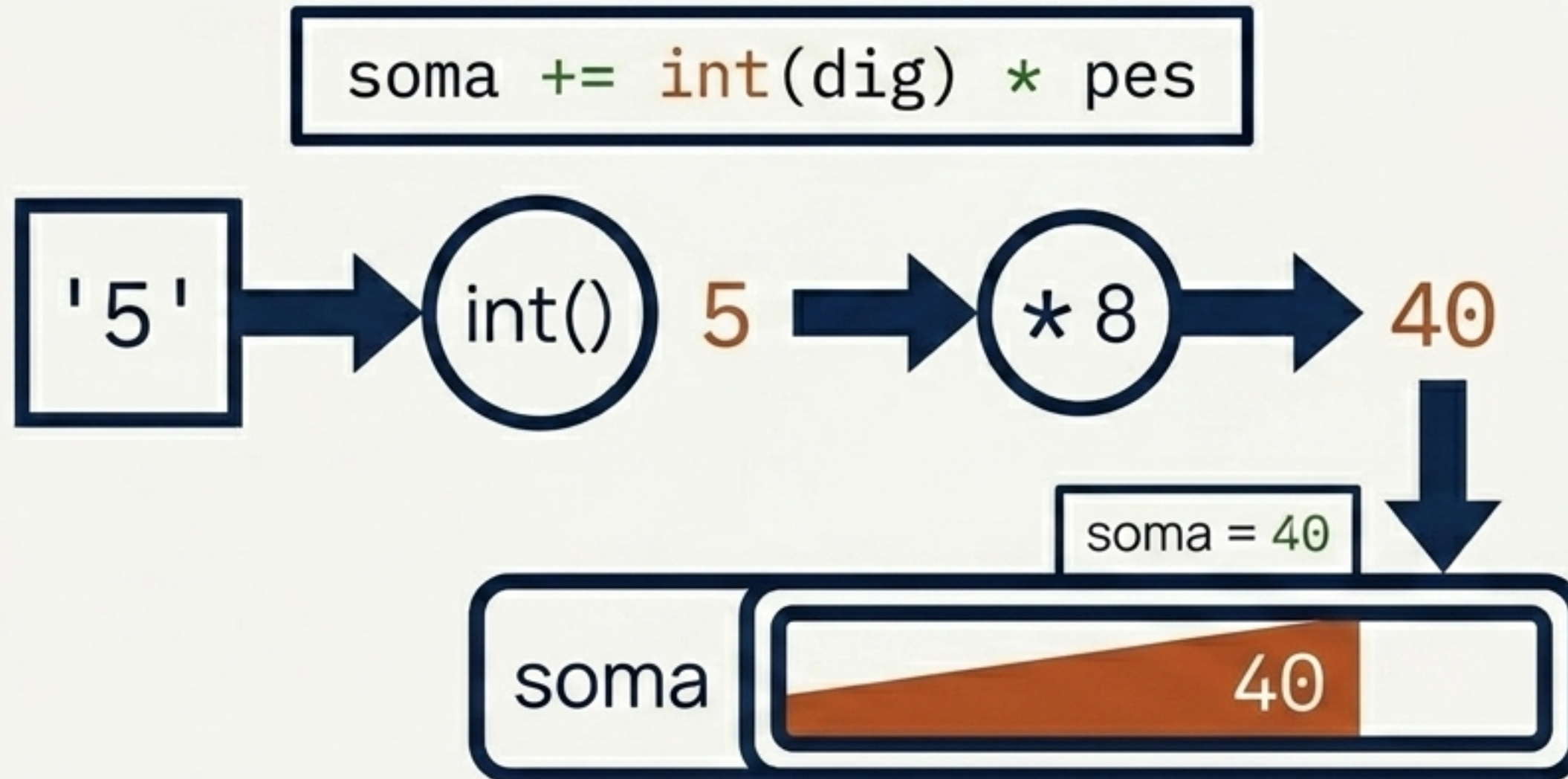
Para calcular a soma, cada dígito deve ser pareado com seu respectivo peso. A função `zip` age literalmente como um zíper mecânico, entrelaçando as duas listas em pares perfeitos para processamento.

```
for dig, pes in zip(digitos, pesos):
```



Conversão e Acúmulo

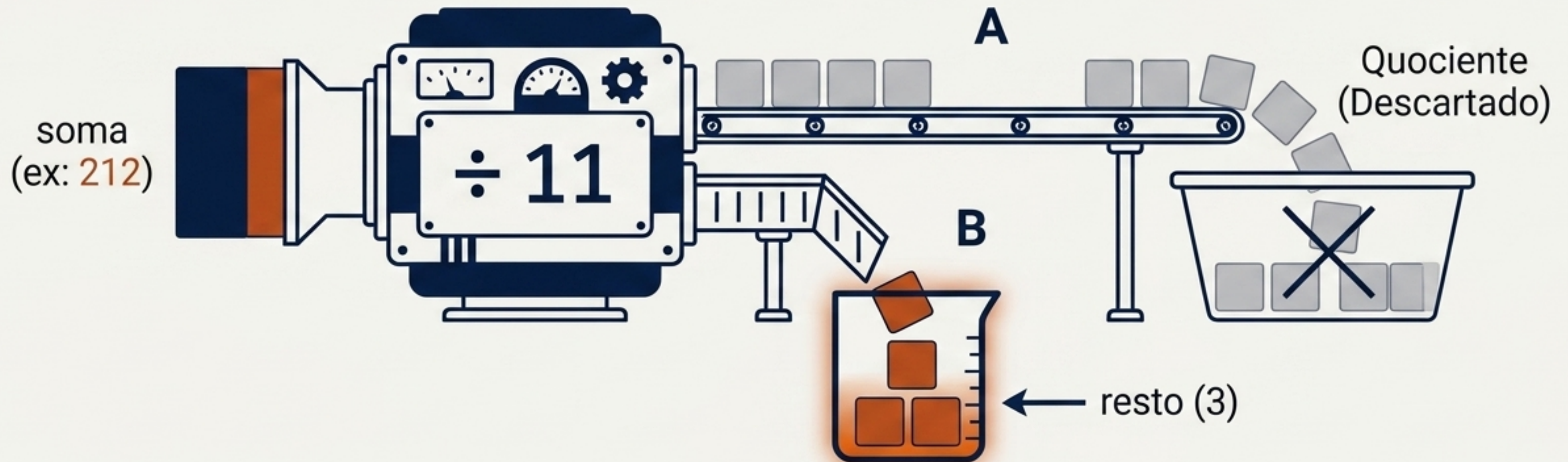
Dentro do loop, o dígito (uma *string*) é convertido em um inteiro. Em seguida, é multiplicado por seu peso emparelhado, e o produto é injetado no acumulador total.



A Lógica do Módulo 11

A criptografia básica do CPF reside no resto de uma divisão por 11. Não nos importamos com o resultado exato da divisão (o quociente), apenas com o que sobra no fundo do balde.

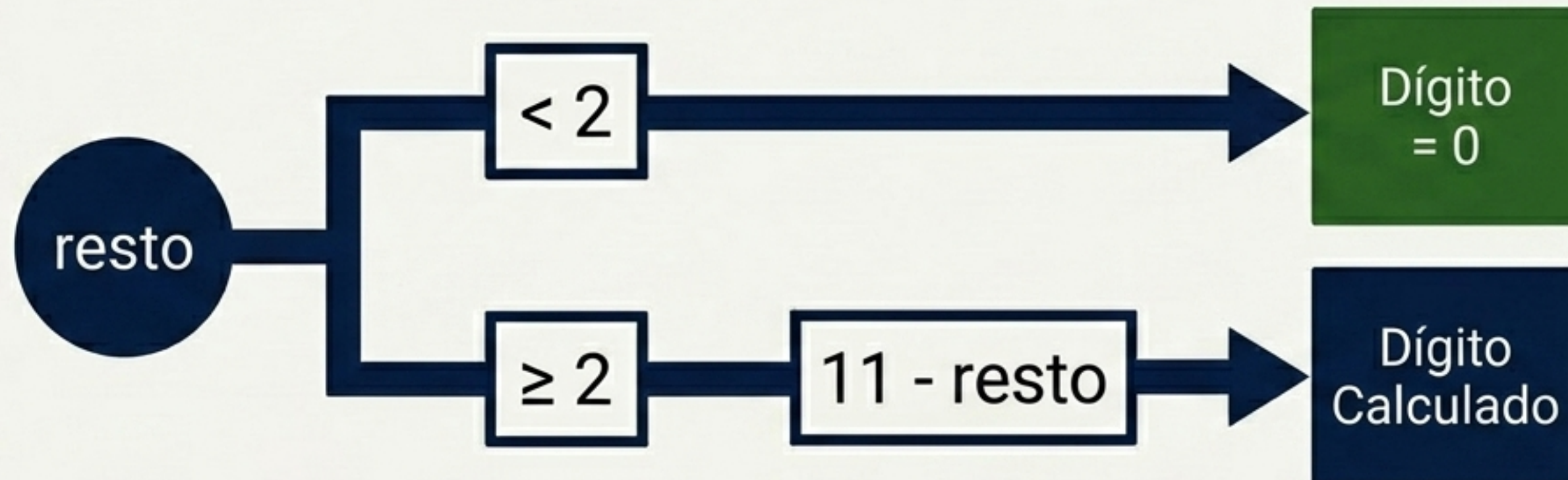
$$\text{resto} = \text{soma} \% 11$$



A Árvore de Decisão Final

O dígito verificador final é decidido pelo valor do resto. Se a sobra for insignificante (menor que 2), o dígito vira zero. Caso contrário, é subtraído de 11 para encontrar o contrapeso exato.

```
return 0 if resto < 2  
else 11 - resto
```



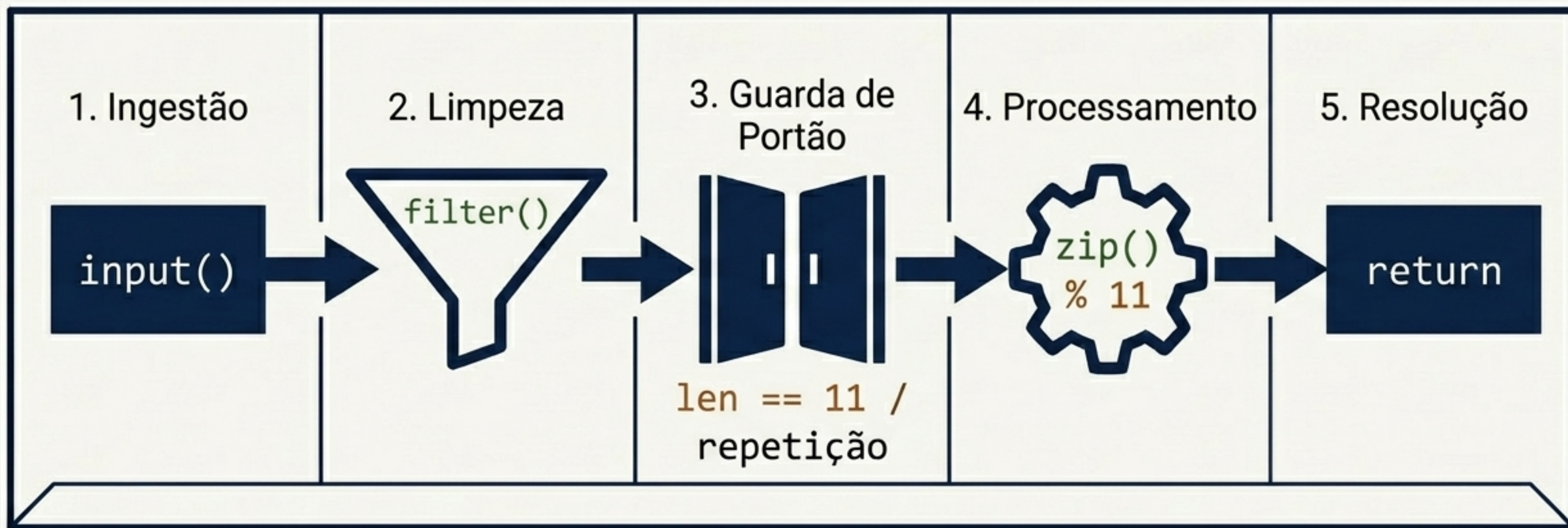
A Matriz de Diagnóstico de Falhas

Por que precisamos de múltiplas camadas de defesa? Diferentes entradas maliciosas ou malformadas exigem armadilhas específicas dentro do algoritmo.

	123.abc... Letras	123 Curto	111.111... Repetido	123...89 Matemática Errada
<code>filter()</code>	✓	•	•	-
<code>len() == 11</code>	•	✓	-	-
<code>cpf[0] * 11</code>	-	-	✓	-
Módulo 11	-	-	-	✓

O Fluxo de Execução Completo

A anatomia completa. O que parece ser um simples script é, na verdade, uma linha de montagem de cinco estágios: Ingestão, Limpeza, Guarda de Portão, Processamento Matemático e Resolução.



O Veredito Final

A força de um algoritmo está em sua capacidade de falhar com segurança. Se o dado bruto não sobreviver a todas as etapas desta linha de montagem, o acesso é negado estruturalmente.

```
else:  
    print("CPF inválido!")
```

```
> Validando 123.456.789-00... [ACESSO NEGADO: CPF inválido!]  
> Validando estrutura de dados... [SISTEMA PROTEGIDO]
```